

Python String Formatting

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

Before Python 3.6 we had to use the `format()` method.

F-string allows you to format selected parts of a string.

To specify a string as an f-string, simply put an `f` in front of the string

```
txt = f"The price is 75 rupees"  
print(txt)
```

```
price = 75  
txt = f"The price is {price:.2f} rupees"  
print(txt)
```

Display the value 75 with 2 numbers after decimal

```
txt = f"The price is {75:.2f} rupees"  
print(txt)
```

Perform a math operation in the placeholder, and return the result:

```
txt = f"The price is {20 * 75} rupees"  
print(txt)
```

Multi operator

```
price = 75
```

```
tax = 18
```

```
txt = f"The price is {price + (price * tax/100)} rupees"
```

```
print(txt)
```

Variables in Python

What are Variables?

Variables are containers that stores data values.

Variables in Python need not to be declared with any particular type, and can even change type after they have been set.

How to creat Variables in Python?

There is no command in Python for declaring a variable.

A variable (of type) is created as soon as you first assign a value to it.

Example

```
x = 25          # This will create a variable x of Integer type having value 25
```

```
y = "Python"    # This will create a variable of String type containing "python"
```

Rules to create a variable in Python:

- 1) A variable name must start with a letter or the underscore character.
- 2) A variable name cannot start with a number.
- 3) A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- 4) Variable names are case-sensitive (name, Name and NAME are three different variables)
- 5) Python keywords cannot be used as variable names.

In Python, there are 33 reserved keywords:

Python Keywords			
False	def	if	raise
None	del	import	return
True	elif	in	try
and	else	is	while
as	except	lambda	with
assert	finally	nonlocal	yield
break	for	not	
class	from	or	
continue	global	pass	

You can see this list any time by typing **help("keywords")** to the Python interpreter. Reserved words are case-sensitive and must be used exactly as shown. They are all entirely lowercase, except for False, None, and True.

Here are some examples of valid variable names:

name

Mobile

_my_address

marks1

marks_2

Marks3

Example

```
x = 25    # x is of type int
```

```
x = "Python" # x is now of type str
```

Type Casting

If you want to specify the data type of a variable, this can be done with casting.

```
x = str(25)  # x will be '25'
```

```
y = int(25)  # y will be 25
```

```
z = float(25) # z will be 25.0
```

Get the Type

You can get the data type of a variable with the `type()` function.

```
x = 25
```

```
y = "Python"
```

```
print(type(x))
```

```
print(type(y))
```

What should be used for String Single Quotes or Double Quotes?

String variables can be declared by using single or double quotes:

Variable names in Python are case-sensitive.

Example

```
a = 25
```

```
A = "Python"
```

```
#A will not overwrite a
```

Data Types in Python

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

Text Type: str

Numeric Types: int, float, complex

Sequence Types: list, tuple, range

Mapping Type: dict

Set Types: set, frozenset

Boolean Type: bool

Binary Types: bytes, bytearray, memoryview

None Type: NoneType

Working with Strings in Python

What is String in Python and How to Concatenate Two String Variables

A string is a sequence of Unicode characters that is enclosed in quotation marks.

```
a = "Hello"
```

```
b = "python"
```

```
print(a + b)
```

String Methods in Python

Case Changing Python String Methods

lower(): **str.lower()**

Converts all uppercase characters in a string into lowercase

upper(): **str.upper()**

Converts all lowercase characters in a string into uppercase

title(): **str.title()**

Convert string to title case

swapcase(): **str.swapcase()**

Swap the cases of all characters in a string

capitalize(): str.capitalize()

Convert the first character of a string to uppercase

List of String Methods in Python

capitalize() str.capitalize()

Converts the first character of the string to a capital (uppercase) letter

casefold() str.casefold()

Converts string to lower case

center()

Pad the string with the specified character.

Ex.: str.center(width[, fillchar])

str.center(40, “ “)

count() str.count("substr")

Returns the number of occurrences of a substring in the string.

startswith(sub, beg, end)

Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring sub

Ex.

```
str = "this is python!!!"  
  
print(str.startswith('this'))  
  
print(str.startswith('is'))
```

endswith()

Returns “True” if a string ends with the given suffix

expandtabs()

Specifies the amount of space to be substituted with the “\t” symbol in the string

Ex.:

```
a = '\tHello Python'  
  
print(a.expandtabs(tabsize=40))
```

find()

Returns the lowest index of the substring if it is found

Ex:

```
str = "this is python!!!"  
  
print(str.find("is"))
```

index()

Returns the position of the first occurrence of a substring in a string

isalnum() **str.isalpha()**

Checks whether all the characters in a given string is alphanumeric **isalpha()**

Returns “True” if all characters in the string are alphabets

isdecimal()

Returns true if all characters in a string are decimal

isdigit()

Returns “True” if all characters in the string are digits

isidentifier()

Check whether a string is a valid identifier or not

islower()

Checks if all characters in the string are lowercase

isnumeric()

Returns “True” if all characters in the string are numeric characters

isprintable()

Returns “True” if all characters in the string are printable or the string is empty

isspace()

Returns “True” if all characters in the string are whitespace characters

istitle()

Returns “True” if the string is a title cased string

isupper()

Checks if all characters in the string are uppercase

join() Returns a concatenated String

```
s = "-"
```

```
seq = ("a", "b", "c") # This is sequence of strings.
```

```
print(s.join( seq ))
```

ljust()

Left aligns the string according to the width specified

Ex:

```
str = "this is python!!!"
```

```
print(str.ljust(40, "."))
```

rstrip()

Returns the string with leading characters removed

replace()

Replaces all occurrences of a substring with another substring

Ex:

```
str = "this is python!!!"
```

```
print(str.replace("is", "is great"))
```

rfind()

Returns the highest index of the substring

rindex()

Returns the highest index of the substring inside the string

rjust()

Right aligns the string according to the width specified

rpartition()

Ex: Split the given string

```
str = "this is python!!!"
```

```
print(str.rpartition(" "))
```

rsplit()

Split the string from the right by the specified separator

Ex:

```
str = "this is python!!!"
```

```
print(str.rsplit("is"))
```

rstrip()

Removes trailing characters

splitlines()

Split the lines at line boundaries

Ex:

```
str1 = "Names:\nA\nB\nC\nD"
```

```
print("String before splitting: " + str1)
```

```
print("String after splitting:")
```

```
print(str1.splitlines())
```

strip()

Returns the string with both leading and trailing characters removed. By default, it removes whitespace such as spaces, tabs, and newlines

translate()

Modify string according to given translation mappings

Ex:

```
l = "tipon"
```

```
e = "12345"
```

```
str = "this is python!!!"
```

```
encoding = str.maketrans(l, e)
```

```
print(str.translate(encoding))
```

Operators

There are 7 types of operators in Python

- 1) Arithmetic operators
- 2) Assignment operators
- 3) Comparison operators
- 4) Logical operators
- 5) Identity operators
- 6) Membership operators
- 7) Bitwise operators

Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation
//	Floor Division

Assignment Operators

Assignment operators are used to assign values to variables

Operator	Example	Value
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	$x == y$
!=	Not equal	$x != y$
>	Greater than	$x > y$
<	Less than	$x < y$
>=	Greater than or equal to	$x >= y$
<=	Less than or equal to	$x <= y$

Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example
And	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Returns False if the result is true	$\text{not}(x < 5 \text{ and } x < 10)$

Identity Operators

Identity operators are used to compare the objects, if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Example:

```
a = 10
```

```
b = 10
```

```
if(b is a):
```

```
    print("b and a are same")
```

```
a = 10
```

```
b = 11
```

```
if(b is not a):
```

```
    print("b and a are not same")
```

Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Example:

```
a="This is Python"
b="This"
if(b in a):
    print("b is a substring of a")
```

```
a="This is Python"
b="Thiz"
if(b not in a):
    print("b is not a substring of a")
```

Conditional Statement

Conditional statements are used to execute different blocks of code based on specific conditions. As the name suggests, If-Else statements offer two paths, allowing for different outcomes depending on the condition evaluated.

Syntax of If Statement in Python

if condition:

Statements to execute if condition is true

Python uses indentation to identify a block. So the block under the Python if statements will be identified

Example

```
i = 26
```

```
if i > 25:
```

```
    print("i is greater than 25")
```

Syntax of If Else in Python

```
if (condition):
```

```
    # Executes this block if condition is true
```

```
else:
```

```
    # Executes this block if
```

```
    # condition is false
```

Example

```
i = 20
```

```
if (i < 15):
```

```
    print("i is smaller than 15")
```

```
else:
```

```
    print("i is greater than 15")
```

```
print("new statement")
```

Nested If Statement

```
if (condition1):
```

```
    # Executes when condition1 is true
```

```
    if (condition2):
```

```
        # Executes when condition2 is true
```

```
        # if Block(2) end here
```

```
    # if Block(1) is end here
```

Example

```
i = 10
```

```
if i < 15:
```

```
print("i is smaller than 15")
```

```
if i < 12:
```

```
    print("i is smaller than 12 too")
```

```
else:
```

```
    print("i is greater than 15")
```

Python Elif

Here, a user can decide among multiple options. The if statements are executed from the top down.

Syntax:

```
if (condition):
```

```
    statement
```

```
elif (condition):
```

```
    statement
```

```
.
```

```
else:
```

```
    statement
```

Example

```
i = 20
```

```
if (i == 10):
```

```
    print("i is 10")
```

```
elif (i == 15):
```

```
    print("i is 15")
```

```
elif (i == 20):
```

```
    print("i is 20")
```

```
else:
```

```
    print("i is not present")
```

One line statement

```
if condition: statement
```

```
if i < 15: print("i is less than 15")
```

```
if condition else statement
```

```
i = 10
```

```
print(True) if i < 15 else print(False)
```


Loops In Python

What is a LOOP

A loop is a sequence of instructions that is continually repeated until a certain condition is reached.

for Loop with range()

In Python, the range() function returns a sequence of numbers.

For example,

```
values = range(4)
```

Here, range(4) returns a sequence of 0, 1, 2, and 3.

Syntax

```
for var in range(value):  
    statement(s)
```

Example

```
# loop from i = 0 to i = 4
```

```
for i in range(5):
```

```
    print(i)
```

range is a function that accepts 3 values

- 1) initial value (starting value)
- 2) condition (ending value)
- 3) step (increment/decrement)

range(start, condition, step)

Example

```
for x in range(1,6):
```

```
    print(x)
```

```
for x in range(1,6,2):
```

```
    print(x)
```

Reverse for loop

```
for x in range(10, 0, -1):
```

```
    print(x)
```

Python for loop with else clause

A for loop can have an optional else clause. This else clause executes after the loop completes.

example

```
for x in range(5,5,-2):
```

```
    print(x)
```

```
else:
```

```
    print("done")
```

Python for loop with break

Example

```
for x in range(10,0,-1):
```

```
    print(x)
```

```
    if x < 5:
```

```
        break
```

```
print("done")
```

Python for loop with continue

Example

```
for x in range(10,0,-1):
```

```
    if x == 5 or x == 3:
```

```
        continue
```

```
    print(x)
```

```
print("done")
```

The pass Statement

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

Example

```
for x in range(5):
```

```
    pass
```

Nested for Loops

Nested loops mean loops inside a loop.

Syntax

Outer_loop Expression:

Inner_loop Expression:

Statement inside inner_loop

Statement inside Outer_loop

Example

```
for x in range(1,3):  
    for y in range(1,3):  
        print(x, y)
```

Output

1 1

1 2

2 1

2 2

Example 2: Printing multiplication table using Python nested for loops

```
for i in range(2, 4):  
    for j in range(1, 11):  
        print(i, "*", j, "=", i*j)
```

nested for with break

```
for i in range(2, 4):  
    for j in range(1, 11):  
        if j == 5:  
            break  
        print(i, "*", j, "=", i*j)
```

nested for with continue

```
for i in range(2, 4):  
    for j in range(1, 11):  
        if j == 5:  
            continue  
        print(i, "*", j, "=", i*j)
```

While Loop

Python While Loop is used to execute a block of statements repeatedly until a given condition is true. When the condition becomes false, the line immediately after the loop in the program is executed.

Syntax

Initial value

while expression:

 statement(s)

 increment

Example

1)

```
number = 1
```

```
while number <= 3:
```

```
    print(number)
```

```
    number = number + 1
```

2)


```
count = 0  
  
while count < 3:  
    count = count + 1  
    print("python")
```

Difference between Python For Loop and Python While Loop

The main difference between Python For Loop Versus Python While Loop is that Python for loop is usually used when the number of iterations is known, whereas Python while loop is used when the number of iterations is unknown

Example

```
# loop is iterated 4 times
```

```
for i in range(4):
```

```
    print(i)
```

```
while True:
```

```
    password = input("Enter password: ")
```

```
    # terminate the loop when user enters exit
```

```
    if password == 'exit':
```

```
        print(f'Status: Entry Rejected')
```

```
        break
```

```
    print(f'Status: Entry Allowed')
```

Python while loop with break statement

We can use a break statement inside a while loop to terminate the loop immediately without checking the test condition. For example,

```
while True:
```

```
    name = input('Enter your name: ')
```

```
    # terminate the loop when user enters end
```

```
    if name == 'exit':
```

```
        print(f'The loop is ended')
```

```
        break
```

```
    print(f'Hi {name}')
```

Python while loop with an else clause

In Python, a while loop can have an optional else clause - that is executed once the loop condition is False. For example,

```
counter = 0
```

```
while counter < 2:
```

```
    print('This is inside while loop')
```

```
    counter = counter + 1
```

```
else:
```

```
    print('This is inside else block')
```

Working with string and slicing

String is a sequence of characters

String indexing

-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
p	y	t	h	o	n		p	r	o	g	r	a	m
0	1	2	3	4	5	6	7	8	9	10	11	12	13

Example

q = "python program"

print(q[4])

#output o

print(q[-2])

#output a

```
print(q[0:7])
```

```
#output python
```

```
print(q[0::2])
```

```
#output pto rga
```

```
Print(q[-1::-1])
```

```
#output reverse string
```

String iteration

In Python, while operating with String, one iterate over the characters of a string .

Example

```
q = "python program"
```

```
for x in q:
```

```
    print(x)
```

Finding length of string

```
q = "python program"
```

```
t = len(q)
```

```
print(t)
```

```
for x in range(t):
```

```
    print(q[x])
```

String reverse using for loop

```
for x in range(t-1,-1,-1):
```

```
    print(q[x])
```

Ord() and Chr() Functions in Python

The ord() function converts a character into its ASCII code, while chr() does the opposite, converting a ASCII code back into its corresponding character.

Syntax

```
print(chr(ascii_code))
```

example

```
print(chr(65))
```

```
#output A
```

```
print(ord("single_character"))
```

Example

```
print(ord("a"))
```

```
#output 97
```